

SKILL WEEK8

TASK-1: TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <string.h>
#include <errno.h>

/*
TASK-1:
To Parse Combined Redirection, Duplicate File Descriptors,
Merge Streams, Verify Ordering, Handle Edge Cases,
Test Integrated Outputs
*/

void test_separate_redirection()
{
    printf("\n--- TEST 1: Separate Redirection ---\n");

    int out_fd = open("stdout.txt",
                     O_WRONLY | O_CREAT | O_TRUNC, 0644);

    int err_fd = open("stderr.txt",
                     O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (out_fd < 0 || err_fd < 0)
    {
        perror("open");
        return;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }
}
```

```

void display_files()
{
    printf("\n--- Generated Files ---\n");

    printf("stdout.txt    -> separate stdout output\n");
    printf("stderr.txt    -> separate stderr output\n");
    printf("merged.txt    -> merged stdout + stderr\n");
    printf("order1.txt    -> ordering test output\n");
}

int main()
{
    printf("=====\n");
    printf(" SKILL WEEK 8 - TASK 1\n");
    printf(" Combined Redirection and File Descriptors\n");
    printf("=====\n");

    printf("\nPID of main process: %d\n", getpid());

    test_separate_redirection();

    test_merge_streams();

    test_ordering();

    test_edge_cases();

    display_files();

    printf("\nAll TASK-1 tests completed successfully.\n");

    return 0;
}

```

```
laasya@laasya: ~/skillweek8
laasya@laasya:~$ mkdir -p skillweek8
laasya@laasya:~$ cd skillweek8
laasya@laasya:~/skillweek8$ nano task1.c
laasya@laasya:~/skillweek8$ gcc task1.c -o task1
laasya@laasya:~/skillweek8$ ./task1
=====
SKILL WEEK 8 - TASK 1
Combined Redirection and File Descriptors
=====

PID of main process: 928

--- TEST 1: Separate Redirection ---
Separate redirection completed.

--- TEST 2: Merge stdout and stderr ---
stdout and stderr were merged into merged.txt

--- TEST 3: Verify Redirection Ordering ---
Correct ordering test completed.

--- TEST 4: Edge Cases ---
Handled invalid file descriptor: Bad file descriptor
Handled file opening error: No such file or directory

--- Generated Files ---
stdout.txt    -> separate stdout output
stderr.txt    -> separate stderr output
merged.txt    -> merged stdout + stderr
order1.txt    -> ordering test output

All TASK-1 tests completed successfully.
laasya@laasya:~/skillweek8$ |
```

TASK-2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <errno.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_LENGTH 512

/*
    SKILL WEEK 8 - TASK 2

    To Parse Complex Commands,
    Combine Pipes and Redirections,
    Resolve Execution Order,
    Validate Syntax,
    Generate Execution Plans,
    Test Scenarios
*/

typedef struct
{
    char command[MAX_LENGTH];
    char *args[MAX_ARGS];

    char input_file[100];
    char output_file[100];
    char error_file[100];

    int input_redirect;
    int output_redirect;
    int error_redirect;
} Command;

```

```

    */
    run_test(
        "Error Redirection",
        "ls wrong_directory 2> error.txt",
        1
    );

    /*
    TEST 5
    Complex pipeline
    */
    run_test(
        "Complex Pipeline + Redirection",
        "cat < sorted.txt | sort | wc -l > count.txt",
        1
    );

    /*
    TEST 6
    Invalid syntax
    */
    run_test(
        "Invalid Syntax",
        "ls | | wc",
        0
    );

    printf("\n=====\\n");
    printf(" ALL TASK-2 TEST SCENARIOS COMPLETED\\n");
    printf("=====\\n");

    return 0;
}
|

```

```

laasya@laasya:~/skillweek8$ nano task2.c
laasya@laasya:~/skillweek8$ gcc task2.c -o task2
laasya@laasya:~/skillweek8$ ./task2
=====
SKILL WEEK 8 - TASK 2
Complex Command Parser
=====

#####
TEST: Simple Pipeline
#####

Command:
ls | wc -l

Parsing Stage 1: ls

Parsing successful.

=====
EXECUTION PLAN
=====

Stage 1:
  Command: ls

Execution order:
  1. Execute Stage 1

Executing command...

merged.txt  order2.txt  stdout.txt  task1.c  task2.c
order1.txt  stderr.txt  task1       task2

Execution completed.

```

TASK-3: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 10

/*
    SKILL WEEK 8 - TASK 3

    To Detect Background Jobs,
    Launch Non-Blocking Processes,
    Return Prompt Immediately,
    Store Job Information,
    Monitor Execution,
    Test Background Tasks
*/

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    int active;
} Job;

Job jobs[MAX_JOBS];

int job_count = 0;

/* Initialize job table */
void initialize_jobs()

```

```
printf("\nMain process PID: %d\n",
      getpid());

initialize_jobs();

/*
   Test 1:
   Detect '&'
*/

test_background_detection();

/*
   Test 2:
   Launch background jobs
*/

run_tests();

/*
   Test 3:
   Monitor jobs
*/

monitoring_demo();

printf("\n=====\\n");
printf(" ALL TASK-3 TESTS COMPLETED\\n");
printf("=====\\n");

return 0;
}
```

```

laasya@laasya:~/skillweek8$ nano task3.c
laasya@laasya:~/skillweek8$ gcc task3.c -o task3
laasya@laasya:~/skillweek8$ ./task3
=====
SKILL WEEK 8 - TASK 3
Background Job Management
=====

Main process PID: 971

=====
BACKGROUND DETECTION TEST
=====

Command: sleep 3 &
Background job detected.
Command after removing &: sleep 3

Command: sleep 5
Foreground job detected.

Command: sleep 8 &
Background job detected.
Command after removing &: sleep 8

=====
BACKGROUND JOB TESTS
=====

Launching background job:
Command: sleep 3
[1] 972 started in background
Parent process continues immediately.

Launching background job:

```

TASK-4: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 10

/*
    SKILL WEEK 8 - TASK 4

    To Create Job Table,
    Store Process Groups,
    Track States,
    Update Status,
    Remove Completed Jobs,
    Validate Job Records
*/

/* Possible job states */
typedef enum
{
    JOB_EMPTY,
    JOB_RUNNING,
    JOB_STOPPED,
    JOB_COMPLETED
} JobState;

/* Job table structure */
typedef struct
{
    int job_id;

```

```
create_test_jobs();

/* Display job table */
display_jobs();

/* Validate records */
validate_all_jobs();

/* Test stopped state */
test_stop_state();

/* Display updated table */
display_jobs();

/* Monitor and remove jobs */
monitoring_demo();

printf("\n=====\\n");
printf(" ALL TASK-4 TESTS COMPLETED\\n");
printf("=====\\n");

return 0;
}
```

```
laasya@laasya:~/skillweek8$ nano task4.c
laasya@laasya:~/skillweek8$ gcc task4.c -o task4
laasya@laasya:~/skillweek8$ ./task4
```

```
=====
SKILL WEEK 8 - TASK 4
Job Table and Process Groups
=====
```

```
Main Process PID: 1000
Main Process PGID: 1000
```

```
Job table initialized.
```

```
=====
CREATING TEST JOBS
=====
```

```
Job added successfully.
Job ID : 1
PID    : 1001
PGID   : 1001
State  : RUNNING
Child PID 1001 created.
Child PGID 1001 created.
```

```
Job added successfully.
Job ID : 2
PID    : 1002
```